

Инструкция для агента: развёртывание стенда ТАКТ на ВМ

Скопируйте текст ниже целиком и передайте агенту, у которого есть SSH-доступ к целевой ВМ.

Инструкция самодостаточна: разворачивает **демо-стенд ТАКТ Industrial Risk Layer** (APM SOC-оператора + синтетический API) в Docker и публикует его. Учтены новые возможности ветки: петля обучения по вердикту, честное состояние канала (SSE + heartbeat + REST-фолбэк),
разделение риск/импакт/доверие, хоткеи триажа с локом и инъекция хаоса.

0. КОНТЕКСТ (что именно разворачиваем)

- Репозиторий: `https://github.com/Shushkary/takt-industrial-risk-layer`
- Ветка: `feature/takt-pt-frontend-dark-ui`
- Связка: `frontend/` (React/Vite, nginx) → проксирует `/api/` на `backend/` (`stand/`, FastAPI).
- Compose: `docker-compose.stand.yml`. Backend :8090, frontend :80 (маппится на :3000).
- **ВАЖНО про состояние.** Движок (`stand/engine_state.py`) держит веса инвариантов, вердикты, локи и аудит **в памяти процесса**. Рестарт контейнера backend = сброс обучения к baseline. Для демо это ок. Есть эндпоинт `POST /api/v1/reset` — вернуть стенд в исходное состояние между показами. Персистентность — отдельная задача (см. `docs/ANTIFRAGILE_BACKLOG.md`, B-1).

1. ДОСТУП К ВМ

Подключитесь к ВМ по SSH (ключ/пользователь/адрес — из ваших учётных данных). Убедитесь, что

есть `sudo` и открыт нужный публичный порт/домен. Если ВМ уже использовалась для этого проекта,

см. также `stand/DEPLOY_PROMPT_ralta.md` (специфика хоста ralta.ru: ключ, 2FA, nginx-префикс `/TAKT_PT`).

2. ПОДГОТОВКА ОКРУЖЕНИЯ

```
sudo apt-get update
# Docker + compose plugin, если не установлены:
command -v docker || curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER" # затем перелогиниться (newgrp docker)
docker compose version || sudo apt-get install -y docker-compose-plugin
```

3. ЗАБРАТЬ КОД

```
sudo mkdir -p /opt/takt && sudo chown "$USER":"$USER" /opt/takt
cd /opt/takt
git clone --branch feature/takt-pt-frontend-dark-ui --depth 1 \
  https://github.com/Shushkary/takt-industrial-risk-layer.git app \
  || (cd app && git fetch origin feature/takt-pt-frontend-dark-ui \
    && git checkout feature/takt-pt-frontend-dark-ui && git pull)
cd /opt/takt/app
```

4. СБОРКА И ЗАПУСК

Определитесь, по какому адресу браузер будет ходить в API стенда, и соберите фронтенд под него.

- **Вариант А — единый origin (рекомендуется, без CORS).** Внешний реверс-прокси отдаёт и статистику фронтенда, и `/api/` с одного адреса. Оставьте относительный API-путь:

```
bash
```

```
docker compose -f docker-compose.stand.yml up -d --build
```

- **Вариант В — API на отдельном адресе.** Укажите публичный адрес API при сборке (CORS в `stand/server.py` уже открыт на `*`):

```
bash
```

```
VITE_TAKT_API_BASE_URL="https://<ваш-домен-или-ip>:8090" \
  docker compose -f docker-compose.stand.yml up -d --build
```

Проверка контейнеров:

```
docker compose -f docker-compose.stand.yml ps
curl -s http://127.0.0.1:8090/health # {"status":"ok","cases":
6,"chaos":"off",...}
curl -s http://127.0.0.1:3000/ | head -c 200 # HTML фронтенда
```

5. ПУБЛИКАЦИЯ ЧЕРЕЗ REVERSE-PROXY (nginx)

Ключевой момент — **SSE**: поток `/api/v1/stream/cases` шлёт именованные события `heartbeat` каждые ~1.5 с. Без отключённого буфера прокси канал будет «висеть» и клиент ложно уйдёт в STALE.

```
# --- Статика/SPA фронтенда APM ---
location / {
    proxy_pass http://127.0.0.1:3000/;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# --- API стенда, включая SSE ---
location /api/ {
    proxy_pass http://127.0.0.1:8090/api/;
    proxy_http_version 1.1;
    proxy_set_header Connection '';
    proxy_buffering off;           # ОБЯЗАТЕЛЬНО для SSE (heartbeat/stream)
    proxy_cache off;
    proxy_read_timeout 86400s;
    proxy_set_header Host $host;
}
```

```
sudo nginx -t && sudo systemctl reload nginx
```

Публикация под префиксом (напр. /ТАКТ_РТ/): собирайте фронтенд с `VITE_TAKT_APP_BASE` = префиксу и проксируйте `/<префикс>/` и `/<префикс>/api/` соответственно. Готовый пример для `ralta.ru` — в `stand/DEPLOY_PROMPT_ralta.md`.

6. ПОСТ-ДЕПЛОЙ ПРОВЕРКА (обязательно — проверяем новые фишки)

Замените `$BASE` на публичный адрес (напр. `https://<домен>` или `http://127.0.0.1:3000` при локальной проверке до прокси). Ниже — smoke-тест API петли обучения, канала и хаоса.

```

BASE="http://127.0.0.1:8090" # для API-проверок берём backend напрямую

# 1) Кейсы отдаются с новыми полями (impact/confidence/tail_risk/invariants)
curl -s "$BASE/api/v1/cases" | head -c 400; echo

# 2) Петля обучения: FP по цепочке снижает риск и каскадит на связанный кейс
curl -s -X POST "$BASE/api/v1/cases/CASE-2026-0731/verdict" \
  -H 'Content-Type: application/json' \
  -d '{"verdict":"fp","reason":"smoke-test","risk_feedback":"too_high","operator":"deploy.check"}' \
  | python3 -m json.tool

# 3) Модель откалибрована (веса < 1.0, есть вердикты)
curl -s "$BASE/api/v1/model" | python3 -m json.tool

# 4) Лок эксклюзивен: второй захват другим оператором → conflict/409
curl -s -X POST "$BASE/api/v1/cases/CASE-20260732/lock" -H 'Content-Type: application/json' \
  -d '{"operator":"op.A"}' >/dev/null
curl -s -X POST "$BASE/api/v1/cases/CASE-20260732/lock" -H 'Content-Type: application/json' \
  -d '{"operator":"op.B"}' | python3 -m json.tool # ожидаем conflict:true / 409

# 5) Аудит — цепочка хэшей непустая
curl -s "$BASE/api/v1/audit" | head -c 300; echo

# 6) Хаос включается и выключается
curl -s -X POST "$BASE/api/v1/chaos" -H 'Content-Type: application/json' -d '{"mode":"drop_source"}' | python3 -m json.tool
curl -s -X POST "$BASE/api/v1/chaos" -H 'Content-Type: application/json' -d '{"mode":"off"}' >/dev/null

# 7) СБРОС стенда в исходное состояние после проверки
curl -s -X POST "$BASE/api/v1/reset" | python3 -m json.tool

```

Проверка в браузере (\$BASE фронтенда):

- Открыть очередь: видны колонки **Импакт / Доверие**, чип «хвост ОТ», индикатор канала вверх (LIVE · SSE), бейдж модели и кнопка **x CHAOS**.
- Хоткеи: / фокусирует поиск; 1–4 меняют серьёзность выбранной строки; a берёт в работу (лок); e эскалирует; x открывает панель хаоса.
- Кейс: открыть карточку — вверх кластер **RISK / ИМПАКТ / ДОВЕРИЕ** и чип хвоста; слева — фальсификаторы и инварианты; внизу — панель вердикта. Выставить FP → появляется карточка «Модель откалибрована» с изменёнными весами и числом затронутых кейсов.
- Честность канала: включить хаос «**Обрыв источника**» → индикатор канала честно уходит в **STALE**, затем в **POLL · резерв** (REST-фолбэк), не показывая «ложный зелёный». Выключить хаос → канал возвращается в **LIVE · SSE**.
- После демо-проверок вызвать `POST /api/v1/reset`, чтобы обнулить внесённые вердикты/локи.

7. КРИТЕРИЙ ГОТОВНОСТИ

Публичный адрес открывается; очередь и карточка кейса работают; SSE держит соединение (в DevTools → Network `stream/cases` висит открытым, приходят `heartbeat`); петля вердикта,

лок (409) и переключение канала под хаосом обрабатывают по проверкам из раздела 6; стенд сброшен в исходное состояние.

8. ЭКСПЛУАТАЦИЯ

```
# Логи
docker compose -f docker-compose.stand.yml logs -f backend
docker compose -f docker-compose.stand.yml logs -f frontend

# Рестарт (ВНИМАНИЕ: сбрасывает обучение – состояние в памяти)
docker compose -f docker-compose.stand.yml restart backend

# Обновление на свежий коммит ветки
cd /opt/takt/app && git pull && docker compose -f docker-compose.stand.yml up -d --build
```

9. ОТКАТ

```
cd /opt/takt/app && docker compose -f docker-compose.stand.yml down
# затем убрать блоки location из конфига nginx и: sudo systemctl reload nginx
```

ЧАСТЫЕ ПРОБЛЕМЫ

- **Канал сразу уходит в STALE/OFFLINE в браузере, хотя backend жив** → на прокси не отключён буфер. Проверьте `proxy_buffering off;` в `location /api/` (раздел 5).
- **Фронтенд грузится, но данных нет / CORS-ошибки** → неверный `VITE_TAKT_API_BASE_URL` при сборке. Пересоберите фронтенд под фактический публичный адрес API и `up -d --build`.
- **404 при прямом заходе на внутренний маршрут SPA** (напр. `/case/...`, `/compare`) → внешний прокси должен отдавать статику через контейнер frontend (его `nginx.conf` уже делает `try_files ... /index.html`); не проксируйте SPA-маршруты мимо него.
- **Порты 1000/2200 на ВМ** — не трогать (могут быть зарезервированы платформой). Стенд их не использует (8090 / 3000).